École Nationale des Sciences Appliquées
Module : Cybersécurité & Investigation Numérique

RAPPORT DE TP
Investigation Forensic

Acquisition des preuves et analyse d'un incident Windows simulé

RAM • KAPE • Prefetch • Registre Windows

Auteur	Nouamane Azzat – Élève Ingénieur
Période	18 – 20 octobre 2026
Environnement	VM Windows 10 « victime » / poste d'analyse Windows
Encadrant	Pr. El Balmany Chawki

Sommaire

1 Contexte, objectifs et périmètre **3**
1.1 Techniques MITRE ATT&CK simulées . . . 3
1.2 Environnement et outils . . . 3

2 Déploiement du framework de simulation (Atomic Red Team) **3**

3 Acquisition des preuves volatiles **4**
3.1 Capture de la mémoire vive (FTK Imager) . . . 4
3.2 État réseau, processus et intégrité (SHA-256) . . . 5
3.3 Chain of Custody – preuve mémoire . . . 6

4 Collecte et triage des artefacts de disque (KAPE) **7**
4.1 Analyse des journaux d'événements (EvtxECmd) . . . 7

5 Analyse des preuves d'exécution (Prefetch) **8**
5.1 Interprétation et précautions . . . 8

6 Inspection de la persistance dans le registre Windows **9**
6.1 Réconciliation des journaux de transactions (*Dirty Hive*) . . . 9
6.2 Analyse de la clé Run utilisateur . . . 9

7 Corrélation des preuves et cartographie MITRE ATT&CK **10**

8 Limites méthodologiques et hypothèses **11**

9 Conclusion et recommandations **11**

A Annexe : état des livrables du TP **12**

Résumé. Ce rapport documente l'acquisition et l'analyse des preuves d'un poste Windows sur lequel le framework *Atomic Red Team* a été déployé pour simuler deux techniques MITRE ATT&CK (T1059.001 et T1547.001). La mémoire vive a été capturée et hachée (SHA-256), l'état réseau et les processus ont été sauvegardés, puis les artefacts de disque ont été collectés avec KAPE et analysés (Prefetch, ruche `NTUSER.DAT`). **Résultat :** l'exécution de PowerShell est établie, mais aucune valeur de persistance « Atomic Red Team » n'a été retrouvée dans la clé `Run` du profil analysé ; ce point est discuté à la section 8.

1 Contexte, objectifs et périmètre

Cette investigation (*Digital Forensics and Incident Response – DFIR*) vise à répondre à la question posée par le TP : *« Que s'est-il passé, et quelles preuves permettent de le démontrer ? »*. Une activité inhabituelle est supposée sur un poste Windows ; il faut déterminer si PowerShell a été utilisé et si un mécanisme de persistance a été introduit.

Le travail suit l'**ordre de volatilité** (RFC 3227) : mémoire vive, puis état réseau et processus, puis artefacts de disque.

1.1 Techniques MITRE ATT&CK simulées

- **T1059.001** – *Command and Scripting Interpreter: PowerShell* (activité d'exécution PowerShell).
- **T1547.001** – *Boot or Logon Autostart Execution: Registry Run Keys* (valeur attendue : `Atomic Red Team` dans `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`).

1.2 Environnement et outils

Rôle	Élément	Outils
Machine cible	VM Windows 10 (profil utilisateur `ilyas`)	Atomic Red Team (simulation)
Collecte	Support USB portable (lecteur `E:`)	FTK Imager, KAPE
Analyse	Poste d'analyse Windows	PECmd, EvtxECmd, Timeline Explorer, Registry Explorer

Table 1: *Architecture du laboratoire.*

2 Déploiement du framework de simulation (Atomic Red Team)

Avant toute simulation, le module `Invoke-AtomicRedTeam` et la bibliothèque de tests (*atomics*) ont été installés dans la VM, depuis une console PowerShell élevée. La figure 1 confirme la fin de l'installation : le dossier des tests est prêt dans `C:\AtomicRedTeam\atomics`.

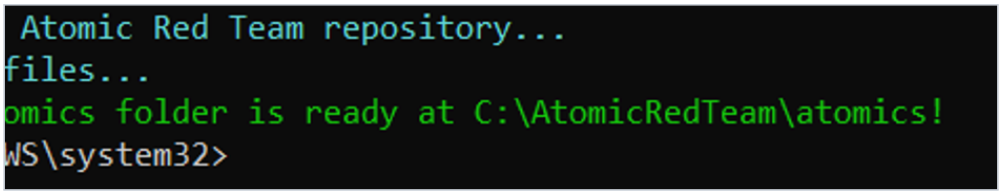


Figure 1: *Fin de l'installation : « Atomics folder is ready at C:\|AtomicRedTeam\|atomics! ».*

La figure 2 liste des chemins de scripts et modules liés au framework (`Invoke-AtomicRedTeam.psm1`, `config.ps1`, `New-Atomic.ps1`, `Invoke-FetchFromZip.ps1`, ...) sous `C:\AtomicRedTeam\invoke-atomicredteam\`. Elle montre aussi le module `powershell-yaml.psm1` sous le profil `ilyas` (`...\OneDrive\Documents\WindowsPowerShell\Modules\`), cohérent avec une installation en portée « utilisateur courant » par ce profil.

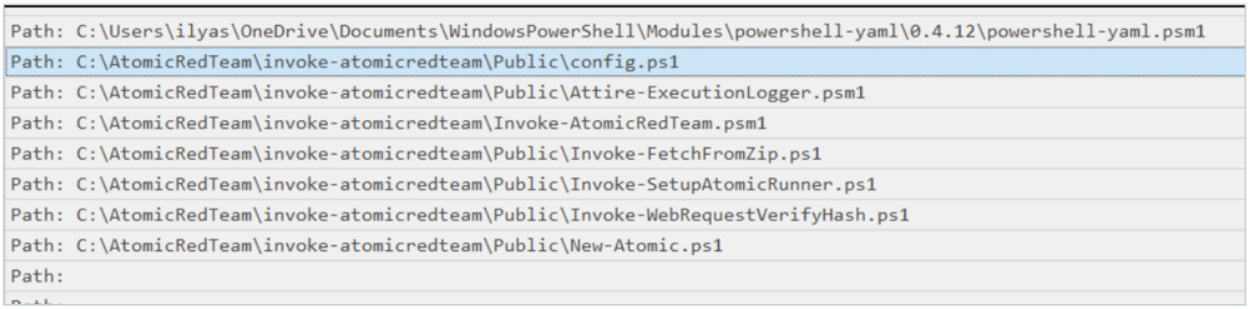


Figure 2: *Traces du framework* `Invoke-AtomicRedTeam` *et du module* `powershell-yaml`.

Ce que ces preuves établissent

Elles prouvent le **déploiement** du framework et que le profil `ilyas` est celui qui l'a installé. Elles ne prouvent pas, à elles seules, qu'un test (`Invoke-AtomicTest`) a été **exécuté** : l'installation et la simulation sont deux étapes distinctes.

3 Acquisition des preuves volatiles

3.1 Capture de la mémoire vive (FTK Imager)

FTK Imager (version 3.1.2.0, lancé depuis le support USB) a été utilisé via *File* → *Capture Memory* avec pour destination `E:\Output\RAM\victime_ram.mem`. La figure 3 montre la capture en cours (262 Mo sur 10 Go, soit 2 %), ce qui indique une mémoire d'environ 10 Go.

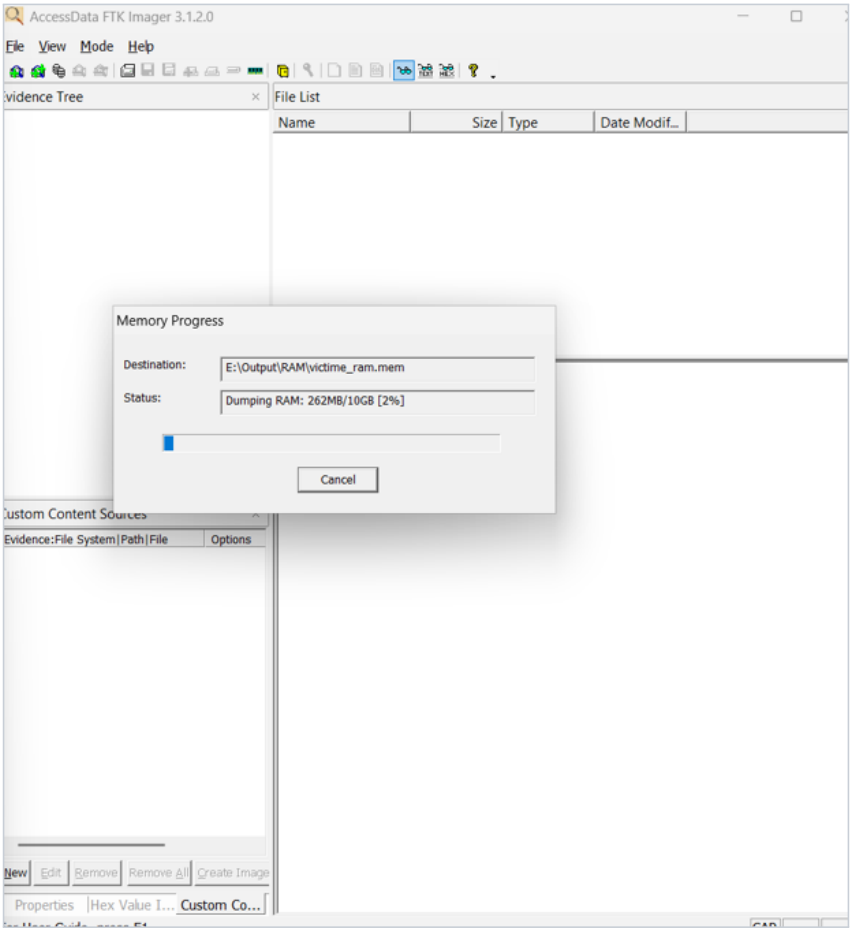


Figure 3: *Capture mémoire en cours dans FTK Imager :* `Dumping RAM: 262MB/10GB [2%]`.

La figure 4 confirme la fin de l'opération (« *Memory capture finished successfully* »). La boîte de dialogue référence alors `pagefile.sys`, ce qui indique que l'option *Include pagefile* était activée : le fichier d'échange est capturé en plus de la RAM.

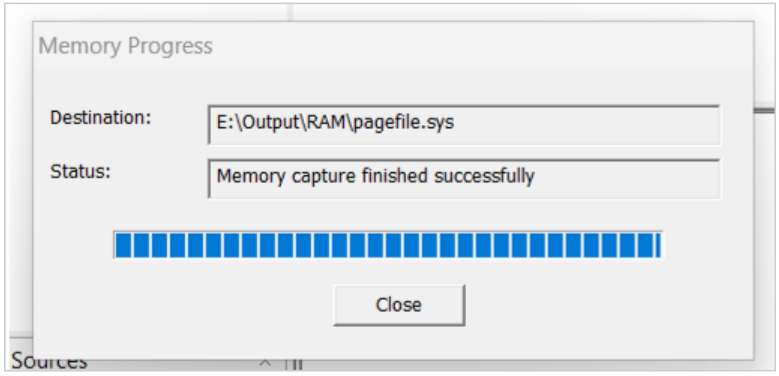


Figure 4: *Fin de la capture mémoire : « Memory capture finished successfully ».*

3.2 État réseau, processus et intégrité (SHA-256)

Immédiatement après la RAM, et toujours machine allumée, l'état réseau et les processus ont été sauvegardés sur le support USB (figure 5) :

Commande	Fichier produit (`E:\Output\RAM\`)
`netstat -anob`	`netstat.txt` (connexions et processus associés)
`arp -a`	`arp.txt` (cache ARP)
`ipconfig /displaydns`	`dnscache.txt` (cache DNS)
`tasklist /v`	`tasklist.txt` (processus en cours)

Le hash SHA-256 du dump a ensuite été calculé avec `Get-FileHash` et enregistré dans `victime_ram.sha256.txt`, puis relu avec `Get-Content`.

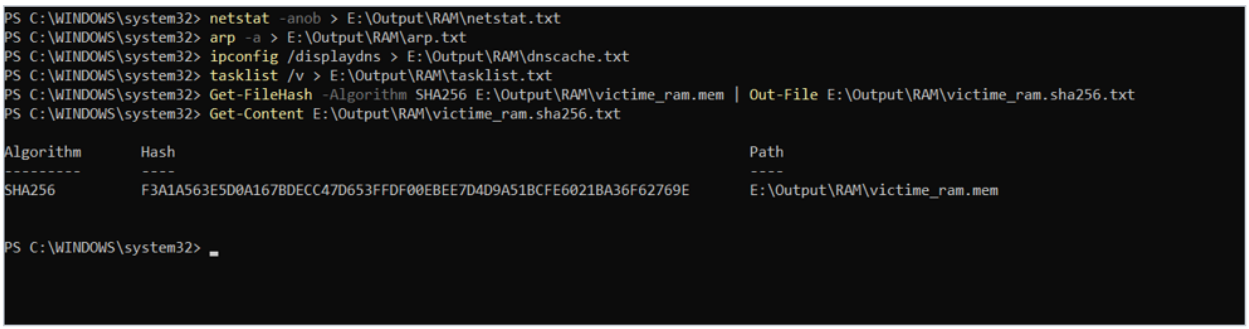


Figure 5: *Sauvegarde de l'état réseau et des processus, puis calcul du hash SHA-256 du dump mémoire.*

Empreinte de référence du dump mémoire

Fichier	`E:\Output\RAM\victime_ram.mem`
Algorithme	SHA-256
Empreinte	`F3A1A563E5D0A167BDECC47D653FFDF00EBEE7D4D9A51BCFE6021BA36F62769E`

3.3 Chain of Custody – preuve mémoire

Champ	Valeur
Evidence ID	EVID-2026-S2-RAM-01
Type de preuve	Dump mémoire brut (`.mem`), pagefile inclus
Source	VM Windows 10 « victime » (RAM ≈ 10 Go)
Outil utilisé	FTK Imager 3.1.2.0 (exécuté depuis le support USB)
Date de capture	20/10/2026 (d'après l'horodatage des captures d'écran)
Hash SHA-256	`F3A1A563E5D0A167BDECC47D653FFDF00EBEE7D4D9A51BCFE6021BA36F62769E`
Destination	`E:\Output\RAM\victime_ram.mem`

Table 2: *Fiche de Chain of Custody de la preuve mémoire.*

4 Collecte et triage des artefacts de disque (KAPE)

La collecte des artefacts a été réalisée avec KAPE vers un répertoire dédié du support d'analyse, distinct de la source, afin de ne rien écrire sur le disque de la victime.

Éléments collectés

Destination	`E:\Output\KAPE TARGET\C\Users\ilyas`
Prefetch	`C:\Windows\Prefetch\*.pf`
Registre	`NTUSER.DAT` et ses journaux de transactions (`.LOG1`, `.LOG2`)

La figure 6 montre le profil `ilyas` après extraction : la ruche `NTUSER.DAT` (2 560 Ko) est présente avec ses journaux `.LOG1` (840 Ko) et `.LOG2` (240 Ko), ce qui permet un rejeu complet des transactions.

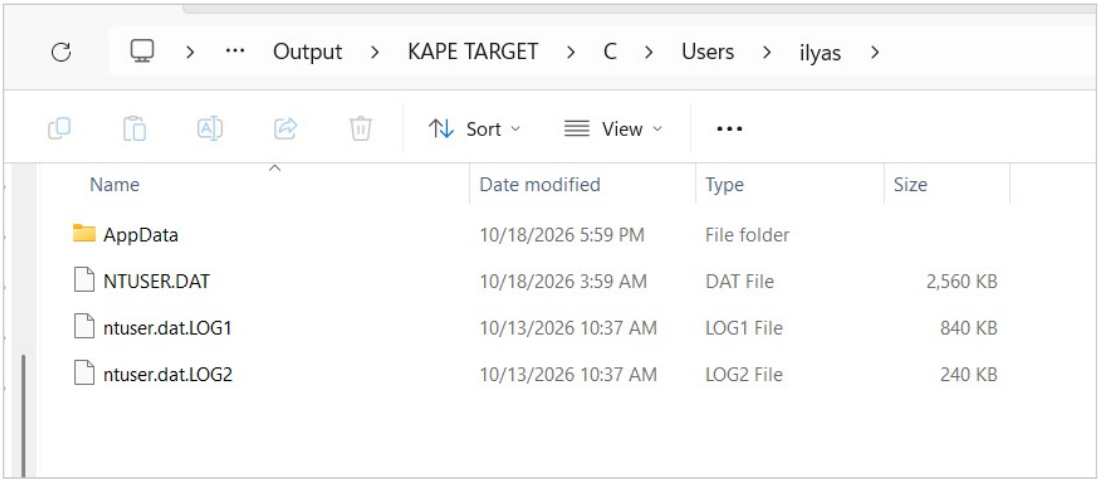


Figure 6: *Artefacts extraits par KAPE et centralisés dans le répertoire d'analyse.*

4.1 Analyse des journaux d'événements (EvtxECmd)

Les journaux d'événements Windows (`.evtx`) collectés ont été convertis en CSV par le module **EvtxECmd** de KAPE (figure 7). Deux sorties horodatées sont présentes (`20261018180625_EvtxECmd_Output.csv` et `20261018202530_EvtxECmd_Output.csv`), exploitables dans Timeline Explorer pour une analyse chronologique.

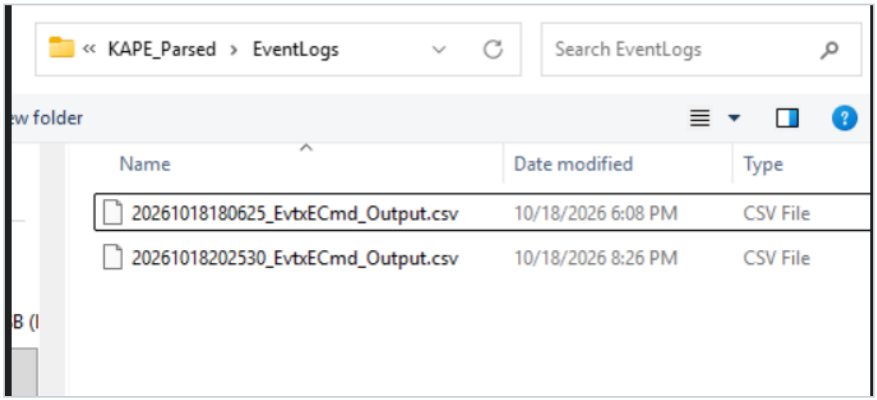


Figure 7: *Sorties CSV d'EvtxECmd dans le dossier* `KAPE_Parsed\EventLogs`.

5 Analyse des preuves d'exécution (Prefetch)

Le mécanisme Prefetch de Windows constitue un indice d'exécution d'un binaire (nom, nombre d'exécutions, horodatages). Le fichier correspondant à PowerShell a été identifié dans le dossier Prefetch collecté (figure 8) puis traité avec `PECmd.exe`.

Artefact identifié

Fichier	`POWERSHELL.EXE-CA1AE517.pf`
Date de modification	18/10/2026 à 19:53 (07:53 PM, heure locale affichée par l'Explorateur)
Taille	61 Ko

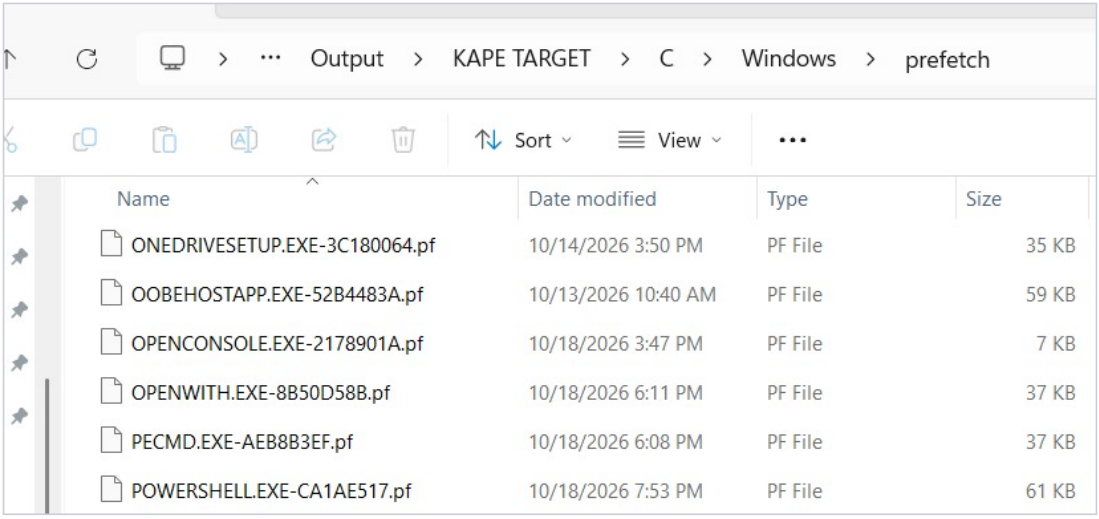


Figure 8: *Dossier Prefetch collecté : présence de* `POWERSHELL.EXE-CA1AE517.pf`.

5.1 Interprétation et précautions

La présence de ce fichier prouve que `powershell.exe` a été exécuté sur le système. Trois précautions s'imposent :

- **PowerShell est un outil courant.** Son Prefetch existe sur la plupart des postes Windows ; il ne suffit pas à lui seul à qualifier une activité malveillante.
- **Activité d'analyse sur la même machine.** Le dossier contient aussi `PECMD.EXE-AEB8B3EF.pf` (18/10/2026, 06:08 PM) : des outils d'analyse ont donc été exécutés sur la machine source, et une partie de l'activité PowerShell peut provenir de l'analyste.
- **Fuseaux horaires.** L'horodatage ci-dessus est l'heure locale affichée par l'Explorateur, alors que Registry Explorer affiche de l'UTC. Pour toute corrélation, relever dans PECmd les valeurs de *Last Run* et *Run Count* (en UTC).

L'attribution de cette exécution au test T1059.001 de la simulation requiert une corrélation supplémentaire : `tasklist.txt`, journaux `.evtx` (Event ID 4104 et 4688) et commandes enregistrées.

6 Inspection de la persistance dans le registre Windows

Les attaquants maintiennent fréquemment un accès en inscrivant un exécutable ou un script dans les clés exécutées à l'ouverture de session (*Run Keys*) – technique T1547.001.

6.1 Réconciliation des journaux de transactions (*Dirty Hive*)

La ruche `NTUSER.DAT` d'un système en fonctionnement reste ouverte en mémoire ; les modifications récentes sont dans les journaux `.LOG1` / `.LOG2`. À l'ouverture, Registry Explorer détecte que les numéros de séquence primaire et secondaire ne correspondent pas (figure 9). Le rejeu des journaux a produit l'état le plus récent de la ruche, enregistré sous `NTUSER.DAT_clean`.

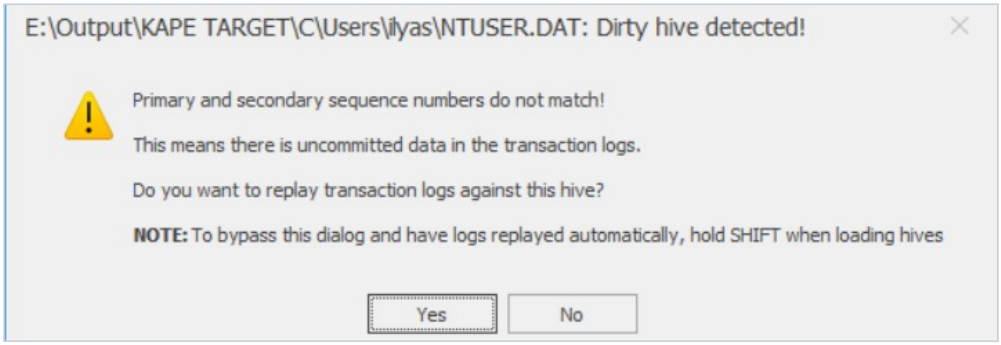


Figure 9: *Détection d'une ruche non clôturée et proposition de rejeu des transactions.*

6.2 Analyse de la clé Run utilisateur

`Software\Microsoft\Windows\CurrentVersion\Run`

Résultats de l'examen

- **Dernier horodatage d'écriture** : 18/10/2026 à 19:38:53 UTC.
- **Valeurs présentes (2) :**
  1. `OneDrive` → `"C:\Users\ilyas\AppData\Local\Microsoft\OneDrive\OneDrive.exe" /background`

2. `MicrosoftEdgeAutoLaunch_...` → `"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe"`

- **Constat :** la valeur attendue `Atomic Red Team` (test T1547.001 n°1) **n'est pas présente** dans la ruche analysée.

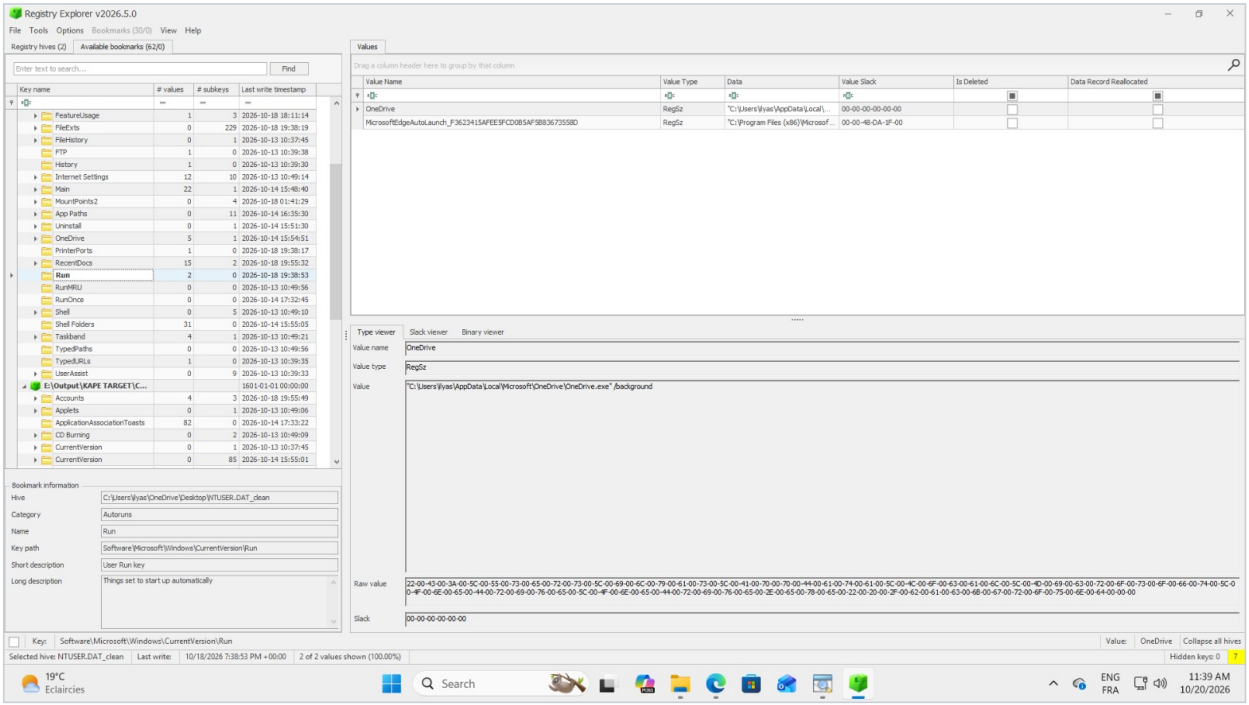


Figure 10: *Clé `Run` dans Registry Explorer : seules deux applications légitimes sont enregistrées.*

7 Corrélation des preuves et cartographie MITRE ATT&CK

Question	Source	Constat
PowerShell a-t-il été utilisé ?	Prefetch `POWERSHELL.EXE-C1AE517.pf` ; `tasklist.txt`	**Oui** : l'exécution est établie par le Prefetch. Le lien avec le test Atomic reste à confirmer (voir §8).
Une persistance a-t-elle été créée ?	`NTUSER.DAT` → `...\Run`	**Non constatée** dans le profil `ilyas` : aucune valeur `Atomic Red Team`.
Les preuves sont-elles intègres ?	SHA-256 de la RAM ; rejeu des journaux de la ruche	Hash du dump calculé et sauvegardé. Ruche reconstruite par rejeu des `.LOG1`/`.LOG2`.
Les artefacts se corroborent-ils ?	Disque + état vivant + registre + framework	**Partiellement** : le framework est installé et PowerShell exécuté, mais la persistance attendue est absente.

Table 3: *Tableau de corrélation des preuves.*

Tactique	ID	Technique	Artefact utilisé	Statut
Execution	T1059.001	Command and Scripting Interpreter: PowerShell	Prefetch (`POWERSHELL.EXE-CA1AE517.pf`)	OBSERVÉ Attribution au test Atomic à confirmer
Persistence	T1547.001	Boot or Logon Autostart Execution: Registry Run Keys	`NTUSER.DAT` → `...\Run`	NON DÉTECTÉ Valeur attendue absente

Table 4: *Correspondance des artefacts avec le référentiel MITRE ATT&CK.*

8 Limites méthodologiques et hypothèses

Absence de la persistance attendue

Le test T1547.001 devait créer la valeur `Atomic Red Team` dans la clé `Run`. Plusieurs explications sont possibles et doivent être départagées :

- le test n°1 n'a pas été exécuté, ou a été exécuté sous un autre compte ou avec une autre numérotation selon la version des *atomics* ;
- un nettoyage (`-Cleanup`) ou une restauration de snapshot a eu lieu avant la collecte ;
- la ruche collectée en direct reflète l'état au moment de la copie (ruche « dirty », rejeu des journaux appliqué, mais valeur absente tout de même).

Autres limites à signaler :

- **Écart temporel.** Les artefacts de disque datent du 18/10/2026, alors que la capture mémoire date du 20/10/2026 ; l'état des processus de `tasklist.txt` ne reflète donc pas nécessairement le moment de la simulation.
- **Processus capturés par l'analyste.** Les commandes de triage ont été lancées depuis une console PowerShell administrateur : `powershell.exe` apparaîtra dans `tasklist.txt` du fait de cette console, pas forcément de la simulation.
- **Version de l'outil.** FTK Imager 3.1.2.0 a été utilisé, alors que le TP recommande une version supérieure ou égale à 4.7.
- **Analyse mémoire.** Le dump `victime_ram.mem` est acquis et haché mais n'est pas analysé ici (Volatility 3 en séance ultérieure).

9 Conclusion et recommandations

1. **Acquisition conforme à l'ordre de volatilité :** RAM (10 Go, pagefile inclus) puis état réseau et processus, avec empreinte SHA-256 du dump enregistrée.
2. **Framework déployé :** `Invoke-AtomicRedTeam` et les *atomics* sont installés sous `C:\AtomicRedTeam\` par le profil `ilyas`.
3. **Exécution de PowerShell établie** par le Prefetch `POWERSHELL.EXE-CA1AE517.pf` (T1059.001) ; son rattachement précis au test Atomic reste à démontrer.
4. **Aucune persistance** `Run` de type `Atomic Red Team` dans le profil `ilyas` (T1547.001) ; la ruche a été reconstruite par rejeu des journaux de transactions.

Recommandations pour la suite

- **Journaux d'événements :** rechercher dans les CSV d'EvtxECmd les Event ID 4104 / 4688, et les chaînes `Invoke-AtomicTest`, `Atomic Red Team` et `Hello, from PowerShell!`, pour dater l'exécution des tests.
- **Image E01 :** charger l'image disque dans FTK Imager, vérifier son intégrité, exporter `NTUSER.DAT` de l'image et comparer avec la ruche issue de KAPE.
- **Autres emplacements de persistance :** `HKLM\...\Run`, tâches planifiées (`C:\Windows\System32\Tasks`), services (`CurrentControlSet\Services`), et autres profils utilisateur.
- **Mémoire :** analyser `victime_ram.mem` avec Volatility 3 (arbre de processus, lignes de commande).

A Annexe : état des livrables du TP

Livrable attendu	État	Remarque
Dump RAM et hash SHA-256	DOCUMENTÉ	Figures 3 à 5
`tasklist.txt` (et netstat, arp, DNS)	PRODUIT	Commandes visibles ; contenu à analyser
Dossier de collecte KAPE	DOCUMENTÉ	Figures 6 et 7
Analyse Prefetch et registre	DOCUMENTÉ	Figures 8 à 10
Tableau de corrélation et conclusion	FOURNI	Sections 7 et 9
Image disque E01 + rapport de vérification FTK	NON DOCUMENTÉ	À joindre si réalisée
Analyse dans FTK Imager (profil, Prefetch de l'E01) et comparaison avec KAPE	NON DOCUMENTÉ	À joindre si réalisée
Snapshot `Before_Atomic_Simulation`	NON DOCUMENTÉ	Capture à ajouter
Exécution des tests (`Invoke-AtomicTest`)	NON DOCUMENTÉ	Capture de la console à ajouter
Chain of Custody	PARTIELLE	Fiche RAM fournie ; autres preuves à compléter

Table 5: *État d'avancement par rapport aux livrables du TP.*